CreatorHQ

Design & Architecture Documentation

Multi-tenant SaaS for Instagram Creators

Version 1.0 • April 2026

Author: Kunta Mallik Raj

Live: https://createhq.netlify.app

Table of Contents

1. Executive Summary
2. System Goals & Non-Goals
3. Architecture Overview
4. Component Design
5. Data Architecture
6. API Design
7. Authentication & Security Architecture
8. External Integrations
9. Deployment Architecture
10. Scalability & Reliability
11. Future Roadmap

1. Executive Summary

CreatorHQ is a multi-tenant SaaS platform built for Instagram creators to consolidate their entire creator business — content analytics, brand-deal pipelines, rate cards, media kits, and benchmarks — into a single workspace.

The system is designed around three principles: tenant isolation enforced at the database layer, a thin orchestration backend that delegates persistence to a managed BaaS, and a clean separation between user-facing CRUD (handled directly by the database client) and computed/external workloads (handled by the API layer).

Key Architectural Decisions

- Backend-as-a-Service first: Supabase provides Postgres, Auth, and RLS, removing the need for a custom user/identity service.
- Stateless FastAPI layer: handles only what cannot be done client-side — Instagram OAuth, post sync, computed analytics, admin email dispatch.
- Edge middleware for route gating: Next.js middleware enforces session checks at the CDN edge before any page renders.
- Defense-in-depth authorization: route middleware → row-level security → JWT verification → role check, in that order.

2. System Goals & Non-Goals

2.1 Functional Goals

Goal	Description
Tenant isolation	Every creator's data is fully isolated; no cross-tenant reads possible even on bugs.
Instagram-first analytics	Pull real engagement metrics from Instagram Graph API; fall back to manual input if not connected.
Deal pipeline	Track brand deals across six lifecycle states from pitch to paid.
Public media kits	Public, shareable creator profiles with a stable slug-based URL.
Rate suggestions	Compute suggested deal rates from follower count and engagement rate.
Admin tester management	Operate inside Meta's app-review constraints by managing testers from an admin panel.

2.2 Non-Functional Goals

Property	Target
Availability	Best-effort on free tier; backend cold-start within ~30s on Render free.
Latency	Page load < 2s on 4G (Netlify CDN-served); API p50 < 400ms warm.
Security	JWT-based auth; RLS on every table; no service key ever shipped to the client.
Cost ceiling	Operate at $0/mo until paid users exist (Supabase free, Render free, Netlify free, SendGrid free).
Time-to-market	Ship a working product without operating a custom auth service or database.

2.3 Non-Goals

- Real-time collaboration — single-user-per-account workspace only.
- Multi-platform analytics — Instagram only in v1; TikTok and YouTube deferred.
- Server-side rendering of media kit PDFs at scale — generated on demand via API.
- Native mobile apps — responsive web only.

3. Architecture Overview

3.1 Layered View

CreatorHQ follows a three-tier architecture with a clean separation between presentation, orchestration, and persistence. External services are integrated as side-cars rather than embedded into the request path.

```
┌──────────────────────────────────────────────────────────────┐
│                    PRESENTATION LAYER                        │
│   Next.js 14 (App Router) • React • TypeScript • Tailwind    │
│   Hosting: Netlify (CDN + Edge Middleware + SSR)             │
└──────────────────────────────────────────────────────────────┘
                 │                          │
                 │                          │
        Direct CRUD via                     │ REST (HTTPS + JWT)
        Supabase JS SDK                     │
                 │                          ▼
                 │              ┌──────────────────────────┐
                 │              │     ORCHESTRATION LAYER  │
                 │              │  FastAPI 0.x • Python 3.11 │
                 │              │  Hosting: Render (Docker) │
```

```
                 │           │                                  │
                 │           │  • Instagram OAuth + sync        │
                 │           │  • Computed analytics            │
                 │           │  • Admin email dispatch          │
                 │           │  • Stripe (planned)              │
                 │           └──────────────────────────────────┘
                 │                          │
                 │                          │
                 ▼                          ▼
          ┌──────────────────────────────────────────┐
          │              PERSISTENCE LAYER           │
          │      Supabase  (PostgreSQL 15 + Auth)    │
          │      • RLS-enforced multi-tenancy        │
          │      • JWT issuance & verification       │
          └──────────────────────────────────────────┘
                                │
                   ┌────────────┼────────────┐
                   ▼            ▼            ▼
   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐
   │   Meta /     │   │   SendGrid   │   │    Stripe    │
   │  Instagram   │   │    Email     │   │  (planned)   │
   └──────────────┘   └──────────────┘   └──────────────┘
```

3.2 Request-Path Patterns

CreatorHQ uses two distinct request paths depending on the operation type:

Path A — Direct DB Access (CRUD)

For straightforward CRUD where the client owns the data shape (deals list, media kit edits, settings), the browser talks directly to Supabase. RLS policies enforce that the JWT-bearing user can only access their own rows.

```
Browser ── Supabase JS SDK ──► Supabase (Postgres + RLS)
                                        │
                                JWT verified, auth.uid()
                                injected into RLS policy
```

Path B — Backend-Mediated (Computed / External)

For operations requiring secrets (Meta App Secret, SendGrid key), heavy computation (engagement rate aggregation), or third-party orchestration, the client calls FastAPI. The backend re-verifies the JWT, performs the work, and uses the service key to write back to Supabase.

```
Browser ── HTTPS + Bearer JWT ──► FastAPI
                                        │
                            1. Decode + verify JWT
                            2. Call external API / compute
                            3. Write via service key
                                        │
                                        ▼
```

```
Supabase
```

3.3 Why a Hybrid Path

- Frontend-direct DB access removes a network hop for the 80% of requests that are simple CRUD.
- Backend mediation is reserved for secrets-handling and computed work, keeping the API surface small.
- Both paths land in the same Supabase instance, so RLS remains the single source of truth for authorization on data reads.

4. Component Design

4.1 Frontend Components

Module	Responsibility
app/(public)	Marketing pages, tools, legal — statically generated, no auth.
app/(auth)	Login, signup, OTP, password reset — uses Supabase Auth client.
app/(dashboard)	Protected pages: dashboard, analytics, deals, rate-card, media-kit, benchmarks, settings, admin.
app/api	Next.js route handlers acting as same-origin proxies to the backend (avoids CORS + hides backend URL).
middleware.ts	Edge middleware running on every request to protected routes; checks Supabase session cookie.
lib/supabase	Two Supabase clients: browser (anon key) and server (cookies-aware for middleware/route handlers).
components/	Reusable UI primitives — built on Tailwind utility classes; no component library.

4.2 Backend Routers

Router	Domain
/instagram	OAuth callback, code-for-token exchange, post sync via Graph API, webhook receiver.
/analytics	Engagement-rate computation, content-type analysis, suggested-rate calculation.
/deals	Brand-deal CRUD with free-tier quota enforcement (max 3).

Router	Domain
/media-kit	Media-kit CRUD plus PDF export endpoint.
/benchmarks	Niche-level benchmark lookups (read-mostly, cacheable).
/waitlist	Tester request submission and admin-only invite dispatch.
/auth	User-info endpoint that decodes the bearer token and returns the canonical user record.

4.3 Service Layer

Inside the backend, business logic lives in services (e.g. an Instagram service handling token exchange, post fetching, and incremental sync). Routers stay thin — they validate input via Pydantic and delegate to services. This keeps HTTP concerns separate from domain logic and makes the services unit-testable.

5. Data Architecture

5.1 Schema Overview

Six core tables form the data model. All user-owned tables foreign-key to users(id) with ON DELETE CASCADE for clean tenant deletion.

Table	Purpose	Owner FK
users	Profile + plan tier (free / pro / agency).	—
instagram_accounts	OAuth-connected IG accounts; one per user typically.	user_id
instagram_posts	Synced post metadata + insights snapshots.	via instagram_accounts
brand_deals	Deal pipeline with six-state status machine.	user_id
media_kits	Public-shareable kits keyed by URL slug.	user_id
tester_requests	Pre-approval Instagram-tester queue.	user_id (text)

5.2 Entity Relationships

```
        ┌──────────┐
        │  users   │
        └──────────┘
             │
             │ 1
             │
┌────────────┼──────────────┬──────────────┐
```

```
   │      │            │             │
   │ 1..1 │ 1..*       │ 1..*        │ 1..*
   ▼      ▼            ▼             ▼
┌─────────────────┐ ┌──────────────┐ ┌──────────────┐
│instagram_       │ │ brand_deals  │ │ media_kits   │
│accounts         │ └──────────────┘ └──────────────┘
└────────┬────────┘
         │ 1
         │
         │ 1..*
         ▼
┌─────────────────┐
│instagram_posts  │
└─────────────────┘
```

5.3 Multi-Tenancy via RLS

Tenant isolation is enforced by Postgres Row Level Security, not application code. Every protected table has an RLS policy keyed on auth.uid() — the user ID extracted from the JWT by Supabase. Even if the client constructs a malicious query, the database refuses to return rows it does not own.

```
-- Pattern: direct ownership
CREATE POLICY "own_deals" ON brand_deals
  FOR ALL USING (auth.uid() = user_id);

-- Pattern: indirect ownership via parent
CREATE POLICY "own_posts" ON instagram_posts
  FOR ALL USING (
    instagram_account_id IN (
      SELECT id FROM instagram_accounts
      WHERE user_id = auth.uid()
    )
  );

-- Pattern: public read with private write
CREATE POLICY "public_kits_read" ON media_kits
  FOR SELECT USING (is_public = true);
CREATE POLICY "own_kit_write" ON media_kits
  FOR ALL USING (auth.uid() = user_id);
```

5.4 Status State Machine — brand_deals

Deals progress through six explicit states. Backwards transitions are allowed (e.g., a deal moves from confirmed back to negotiating).

```
pending ──► negotiating ──► confirmed ──► in_progress ──► paid
   │             │              │
   └─────────────┴──────────────┴──► cancelled (terminal)
```

6. API Design

6.1 Conventions

- All endpoints are namespaced by domain — /instagram, /analytics, /deals, /media-kit, /benchmarks, /waitlist, /auth.
- Authorization uses bearer JWTs in the Authorization header. Tokens are issued by Supabase Auth and verified using the shared JWT_SECRET.
- Request and response payloads are validated by Pydantic models, returning 422 with structured errors on bad input.
- CORS is locked to a single allowlisted origin (the Netlify domain). Preflights are handled by FastAPI's CORS middleware.

6.2 Endpoint Catalog

Instagram

Method	Path	Auth	Purpose
GET	/instagram/webhook	—	Meta verification challenge.
POST	/instagram/webhook	—	Receive subscription events (signed by Meta).
GET	/instagram/auth/callback	—	OAuth code exchange; persists access token.
POST	/instagram/sync/{user_id}	JWT	Pull latest posts into instagram_posts.

Analytics

Method	Path	Auth	Purpose
GET	/analytics/engagement/{user_id}	JWT	Mean engagement rate + breakdowns.
GET	/analytics/content/{user_id}	JWT	Per-post insights with format ranking.
GET	/analytics/rates/{user_id}	JWT	Suggested deal rates.

Deals & Media Kits

Method	Path	Auth	Purpose
GET / POST	/deals/	JWT	List or create deals; quota enforced for free tier.
PATCH / DELETE	/deals/{deal_id}	JWT	Mutate or remove a deal.

Method	Path	Auth	Purpose
GET / POST	/media-kit/	JWT	List or create media kits.
PATCH	/media-kit/{kit_id}	JWT	Update kit content or visibility.
GET	/media-kit/{kit_id}/pdf	JWT	Render kit as PDF.

Admin & Health

Method	Path	Auth	Purpose
POST	/waitlist/request-tester	JWT	User submits IG handle for tester approval.
POST	/waitlist/send-invite	JWT (admin)	Send activation email via SendGrid.
GET	/health	—	Liveness probe.

7. Authentication & Security Architecture

7.1 Defense-in-Depth Layers

Authorization is enforced at four independent layers. A bypass of any single layer does not grant access — an attacker must defeat all four.

```
┌──────────────────────────────────────────────────────────┐
│ Layer 1 — Edge Middleware (Next.js)                      │
│   Runs on Netlify edge. Checks Supabase session cookie.  │
│   No session → redirect to /login. No DB query yet.      │
├──────────────────────────────────────────────────────────┤
│ Layer 2 — Row Level Security (Postgres)                  │
│   Every table policy keyed on auth.uid().                │
│   Wrong user → zero rows returned, regardless of query.  │
├──────────────────────────────────────────────────────────┤
│ Layer 3 — JWT Verification (FastAPI)                     │
│   Bearer token decoded with JWT_SECRET on every request. │
│   Missing / expired / invalid → 401 Unauthorized.        │
├──────────────────────────────────────────────────────────┤
│ Layer 4 — Role Check (Application)                       │
│   Admin endpoints compare token email against admin list.│
│   Non-admin → 403 Forbidden, even with valid JWT.        │
└──────────────────────────────────────────────────────────┘
```

7.2 Auth Flows

Email + Password

The browser calls Supabase Auth directly. On success, Supabase sets an HTTP-only cookie containing the session and returns access + refresh tokens. The middleware reads this cookie on subsequent navigation.

Magic Link / OTP

Used as a passwordless alternative. Supabase emails a six-digit code; the client calls verifyOtp to exchange it for a session.

Password Reset

resetPasswordForEmail sends a recovery link pointing to /api/auth/callback?type=recovery. The callback exchanges the code for a session and redirects to /reset-password where the user submits a new password via updateUser.

7.3 Secrets Handling

- The Supabase service key never leaves the backend — it is only present in Render environment variables.
- The Instagram App Secret is held only by the backend; the frontend uses the public App ID for the OAuth redirect.
- The SendGrid API key is backend-only; user-facing email triggers go through authenticated endpoints.
- Browser code uses only the Supabase anon key, which is RLS-gated and safe to expose.

8. External Integrations

8.1 Instagram (Meta Graph API)

CreatorHQ uses Instagram Business Login — Meta's OAuth flow for Professional accounts. Until the Meta app review is approved, only manually-added testers can connect.

OAuth Sequence

```
User                Browser             Backend            Instagram
 |                    |                    |                   |
 | click "Connect"    |                    |                   |
 |──────────────────→ | redirect to IG OAuth                   |
 |                    |──────────────────────────────────────→|
 | authorize          |                    |                   |
 |──────────────────────────────────────────────────────────→
 |                    | ←──────── redirect back with code ──────|
 |                    | /api/instagram/callback?code=xxx        |
```

```
  │                    │──────────────────►│                    │
  │                    │                   │ POST /oauth/access_token
  │                    │                   │──────────────────►│
  │                    │                   │ ◄──────── token ──│
  │                    │                   │ INSERT instagram_accounts
  │                    │                   │ POST /sync         │
  │                    │                   │ fetch /me/media ──►│
  │                    │                   │ ◄────── posts ────│
  │                    │                   │ INSERT instagram_posts
```

Tester Workflow

- User submits Instagram username from /settings.
- Backend writes to tester_requests and emails the admin via SendGrid.
- Admin manually adds the username in the Meta Developer portal.
- Admin clicks Send Invite in /admin — backend dispatches activation email and flips status to invited.
- User accepts the tester invite on Instagram, then completes the Connect Instagram flow.

8.2 SendGrid

Two transactional templates are dispatched: an admin notification on tester request, and a user activation email on admin approval. The free tier allows 100 emails/day, sufficient for the early-access phase.

8.3 Stripe (Planned)

Subscription billing will be added via Stripe Checkout. The product is provisioned at $5/mo recurring (price_1TQVcEPmTjflkcMOqlLl8JvJ). The webhook handler is the source of truth for plan upgrades — never the redirect URL — to prevent client-side tampering.

```
Browser ──► /payments/create-checkout ──► Stripe Checkout
                                           │
                                           │ user pays
                                           ▼
                      /payments/webhook ◄── Stripe (signed)
                              │
                              │ verify signature
                              │ UPDATE users.plan = 'pro'
                              ▼
                         Supabase
```

9. Deployment Architecture

9.1 Hosting Topology

Tier	Provider	Plan	Notes
Frontend	Netlify	Free	CDN-served Next.js with Edge Middleware; auto-deploy from main.
Backend	Render	Free (Docker)	0.1 CPU / 512MB; cold-starts after 15min idle. Upgrade path: Starter $7/mo.
Database	Supabase	Free	500MB storage, 50K MAU, daily backups.
Email	SendGrid	Free	100/day from a verified single sender.

9.2 CI/CD Pipeline

```
developer pushes to main
         │
         ├──► Netlify
         │      ├── npm install
         │      ├── next build
         │      └── deploy to global CDN  (~90s)
         │
         └──► Render
                ├── docker build
                ├── push image
                └── rolling deploy        (~3min)
```

9.3 Environment Boundaries

- Frontend env vars prefixed NEXT_PUBLIC_* are baked into the client bundle — only public values.
- Backend env vars are runtime-only and never exposed to the browser.
- Supabase JWT_SECRET is shared between Supabase and the backend so token verification works without an extra round trip.

10. Scalability & Reliability

10.1 Current Bottlenecks

Bottleneck	Cause	Mitigation
Backend cold-starts	Render free tier spins down after 15min.	Upgrade to Starter $7/mo, or add a periodic warm-up ping.

Bottleneck	Cause	Mitigation
Instagram rate limits	Graph API caps per-user calls per hour.	Incremental sync (pull only new posts); cache responses.
SendGrid 100/day cap	Free tier limit.	Move to paid plan when approval volume exceeds limit.
Supabase 500MB	Free-tier storage cap.	Archive cold post data to object storage when nearing cap.

10.2 Horizontal Scaling Path

- Frontend already scales horizontally via Netlify's CDN; no work needed.
- Backend is stateless — moving to multiple Render instances behind a load balancer requires only a plan upgrade.
- Database can vertically scale on Supabase (Pro tier) and supports read replicas at higher tiers.

10.3 Failure Modes & Handling

Failure	Impact	Behavior
Backend down	Instagram sync, computed analytics, admin emails fail.	CRUD via Supabase still works; UI shows degraded state.
Supabase down	Auth + all data unavailable.	App unusable; status page link surfaced.
Instagram API 429	Sync fails for that user.	Exponential backoff; user sees Last synced timestamp unchanged.
SendGrid failure	Admin invite email not delivered.	Status not flipped to invited; admin can retry.

11. Future Roadmap

11.1 Near-Term

- Stripe subscription billing — endpoints scaffolded, webhook handler pending.
- Meta App Review submission — unlocks public sign-up beyond testers.
- Background job queue for sync — replace per-request sync with a scheduled worker.

11.2 Mid-Term

- TikTok and YouTube Shorts integration via their respective Creator APIs.
- Email outreach module — draft and send pitch emails to brands directly from the deal pipeline.

- Team workspaces — agency-tier accounts with multiple managed creators.

11.3 Long-Term

- AI-driven content recommendations using historical engagement data.
- Marketplace surface where brands discover creators by niche and benchmark percentile.
- Native mobile clients (iOS first), reusing the same FastAPI backend.

End of document